# Компиляторы. Введение

Косарев Дмитрий

6 сентября 2025 г.

Дата сборки: 5 сентября 2025 г.



# В этих слайдах

1. Административные вещи

2. Архитектура

3. Чего почитать?

# В этом курсе: компиляторы

(и, возможно, недорассказанное про ФП)

Самостоятельная работа (разработка компилятора miniML) + экзамен формулируют оценку

- Синтаксический анализ + вывод типов
- Трансляция в ANF форму
- Кодогенерация с использованием LLVM
- Кодогенерация в ассемблер (amd64, RISCV)
- Сборка мусора при кодогенерации в amd64

Самостоятельная работа групповая (1–2 человека)

Оценки ECTS за курс ФП у членов группы не должны отличаться больше чем на 2 буквы

# Немного истории

- В 1950г. программировали на машинных кодах и на ассемблере
  - Медленно и дорого
- В 1954 Д. Бэкус оттранслировал язык выражений в код мэйнфрейма IBM 794[1]
- Проект Fortran I (FORmula TRANslator) — первый компилятор

Последствия:

- Многие программисты опасались (опасаются), что языки высокого уровня ухудшают производительность
- Первый оптимизирующий компилятор
- Фортран оказали влияние на другие компиляторы
  - Многие в него компилировались
  - Делали архитектуру компилятора как в Фортране

[1] https://en.wikipedia.org/wiki/IBM_704

# Исполнение программ

Языки программирования описывают абстракции для проведения вычислений

- Например, поток управления и абстракции данных
- Языки высокого уровня добавляют: быстрое прототипирование, продуктивность, поддерживаемость и отлаживаемость

Эти абстракции надо преобразовывать в язык, понятный машине

# Что такое компилятор

**Определение**

Компилятор — это системное ПО, которое транслирует программы с исходного языка программирования в целевой язык

Типичные исходные языки: C, OCaml, C#...
Целевой (англ. target) язык — обычно инструкции процессора

Важные свойства компилятора:

- Порождать *корректный* код
- *Улучшать* код согласно некоторой метрике
- *Помогать* пользователю: сообщать о настоящих и потенциальных ошибках

# Трансляторы из исходного кода в исходный код

Порождают код на другом языке программирования, вместо ассемблера процессора

- Ещё называют транскомпиляторы или transpiler'ы
- Например, Typescript $\rightsquigarrow$ Javascript, или какой-нибудь модельный язык $\rightsquigarrow$ Си

Чтобы исполнять скомпилированное, нужны ещё фазы трансляции

Антипример: LaTeX $\rightsquigarrow$ PDF обычно считают компиляцией

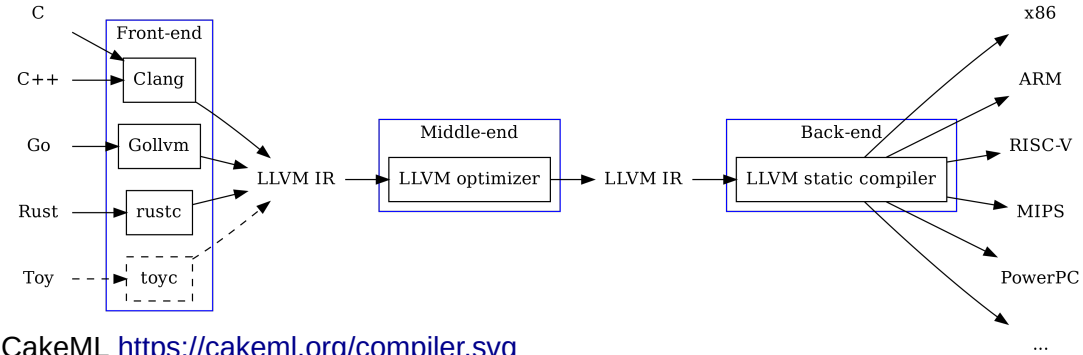
| Transpiler | Compiler |
|---|---|
| Конвертирует между языками с примерно одинаковым уровнем абстракции | Традиционный компилятор транслирует из языка высокого уровня в низкий |

# Много чего уже было на первом курсе

- Интерпретаторы
- Гибридные схемы (LISP, JVM, байткод, JIT)
- Некоторые примеры оптимизаций

# Архитектура компиляторов

- Однопроходные и многопроходные
- LLVM (многопроходный, retargetable)




- CakeML https://cakeml.org/compiler.svg
- Mlton (Whole program optimization) http://mlton.org/CompilerOverview

[1] https://blog.gopheracademy.com/postimages/advent-2018/llvm-ir-and-go/llvm_compiler_pipeline.png

# Варианты архитектур

*Монолитная*

- Потенциально эффективнее, менее гибкая

*Многопроходная (nanopass)*

- Менее сложная
- Компилируется дольше

# Нельзя просто взять и скомпилировать

Трансляция «в лоб» не всегда возможна

- Проблемы с абстракцией: абстрактные типы данных и переменные vs. память и регистры
- Поток управления vs. jump’ы

Некоторые языки дальше от железа, чем другие (например, ООП vs. Си)

Как действуют:

- Транслируют по шагам, где каждый отвечает за простую, четко поставленную задачу
- Получается серия промежуточных представлений (англ. intermediate representation, IR) для хранения информации. Представления можно переписывать для совершения оптимизаций
- По ходу трансляции IR становятся более похожи на машинный код, чем до этого

# Части компилятора: Фронтенд (англ. frontend)

«Олдскульный» подход: лексер и парсер какой-то грамматики

**Лексер** (англ. lexer, лексический анализатор) разбивает вход на слова (англ. token)

**Парсер** (англ. parser, синтаксический анализатор) объединяет слова в представление программы — дерево абстрактного синтаксиса

**Генераторы** синтаксических анализаторов: по грамматике дают парсер (ANTLR, Menhir)

**Scannerless** Можно и без лексера

**Вручную** рекурсивным спуском

# Части компилятора: Семантический анализ, оптимизации

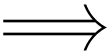
Типизация обычно здесь, но можно и во frontend

Разрешение имён

IR много оптимизаций, не зависящих от архитектуры

Например,

```
id1 = id2 + id3 * 60
```



```
t1 = int_of_float(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

# Оптимизации — это сложно

Для разных программ полезны разные оптимизации
(В идеале) Компилятор должен «настраиваться» под программы

- Выбор и порядок применения оптимизаций
- Параметры полезности оптимизаций (например, граница размера функции для inlining)
- Autotuning — автоматический перебор опций компиляции, чтобы получить получше результат
- Некоторые применяют ML

# Части компилятора: Backend. Выбор инструкций

- Преобразование IR в машинные инструкции
- Специфично для конкретного процессора
- Предполагает бесконечное количество *виртуальных регистров*

```
y = y * 81
```



```
li a0, 81
mul y, y, a0
```



```
# y:= y<<3+y = y*9
sh3add y, y, y
# y:=  y*9
sh3add y, y, y
```

# Части компилятора: Backend

Распределение регистров (англ. register allocation)

- Нужно решить какие значению будут находиться в небольшом количестве регистров процессора

Планирование инструкций (англ. instruction sheduling)

- Перестановка инструкций, чтобы максимизировать использование вентелей железа и уменьшать время исполнения в тактах

# Планирование инструкций. Пример

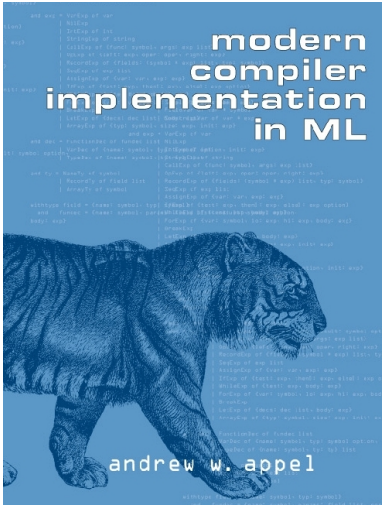
```
LOAD @ADDR1, @OFF1 -> R1          LOAD @ADDR1, @OFF1 -> R1
ADD R1, R1 -> R1                  LOAD @ADDR2, @OFF2 -> R2
LOAD @ADDR2, @OFF2 -> R2          LOAD @ADDR3, @OFF3 -> R3
MUL R1, R2 -> R1          ⟹      ADD R1, R1 -> R1
LOAD @ADDR3, @OFF3 -> R2          MUL R1, R2 -> R1
MUL R1, R2 -> R1                  MUL R1, R3 -> R1
STORE R1 -> @ADDR1, @OFF1         STORE R1 -> @ADDR1, @OFF1
```

В предположении:

- LOAD/STORE — 3 такта
- MUL — 2 такта
- ADD — 1 такт

# Чего почитать?

- «Tiger book» [1]
- «Dragon book» [2]
- «Advanced compiler design and implementation» [3] Про оптимизации



# Ссылки & Acknowledgements


- Andrew W. Appel. *Modern Compiler Implementation in ML*. 1997.
- Alfred V. Aho и др. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. 2006.
- Steven S. Muchnick. *Advanced compiler design and implementation*. 1998.